



数组

北京理工大学计算机学院
金旭亮

Arrays (数组)

- 一种简单的数据结构
- 元素具有相同的数据类型
- 一旦创建之后，尺寸保持不变
- 元素在内存中连续分布

c代表数组名

c.length得到c这个数组的长度

c有12个元素 (c[0], c[1], ... c[11])

c[0]这个元素的值是 -45

"[]"在java中拥有最高优先级

c代表数组名，
通过它可以找到
所有的数组元素

每个元素对应着一个整数值，
标明它在数组中的精确位置

c[0]	-45
c[1]	6
c[2]	0
c[3]	72
c[4]	1543
c[5]	-89
c[6]	0
c[7]	62
c[8]	-3
c[9]	1
c[10]	6453
c[11]	78

一个拥有12个元素的数组在内存中的分布图

数组的下标

- Subscript（下标）
 - 亦称为 index（索引），代表元素在整个数组中的位置
 - 必须是整数或整数表达式，例如

```
int a = 5;  
int b = 6;  
c[ a + b ] += 2;
```

数组的创建方法



数组是对象，需要占用内存空间。

以下代码声明一个数组变量并为之分配内存：

```
int[] c = new int[ 12 ];
```

上述代码相当于

```
int[] c;           // 声明数组变量  
c = new int[ 12 ]; // 分配内存
```

上述代码创建的是一个“原始数据类型”的数组，我们还可以创建“对象数组”：

```
String[] b = new String[ 100 ];
```

数组初始化

- 数组创建之后，可以使用循环或赋值语句对其进行初始化。
- Java提供了一种使用初始化列表的“简便方法”，使用它可以省去使用 new 关键字创建数组对象的工作：

```
int[] n = { 10, 20, 30, 40, 50 };
```

数组初始化及遍历输出示例

```
public class InitArray {  
    public static void main(String[] args) {  
        //声明数据变量并分配内存, 同时初始化数组元素  
        int[] n = {32, 27, 64, 18, 95, 14};  
  
        var output = new StringBuilder("下标\t值\n");  
        //使用StringBuilder循环构建输出字符串  
        for (int i = 0; i < n.length; i++)  
            output.append(i + "\t" + n[i] + "\n");  
        System.out.println(output);  
  
        //另一种初始化方式  
        int[] arr = new int[5];  
        //使用固定值填充  
        Arrays.fill(arr, val: 1);  
        //连接所有元素, 构成一个字符串  
        String arrContent = Arrays.toString(arr);  
        //输出: [1, 1, 1, 1, 1]  
        System.out.println(arrContent);  
    }  
}
```



下标	值
0	32
1	27
2	64
3	18
4	95
5	14
[1, 1, 1, 1, 1]	

示例: InitArray.java

数组元素必须具有相同的数据类型？

请看以下代码：

```
public static void main(String[] args)
{
    Object[] arr={"a","b",13};
    System.out.println(arr);
}
```

上述代码可以顺利通过编译，并且输出一个“很奇怪”的结果：

[Ljava.lang.Object;@ba8a1dc

← 这是什么东东？

为什么会这样？学到后面就明白了，此处先不求甚解！



有些事，做了才知道。
有些事，錯了才知道。
有些事，長大了才知道。

两种不同类型的数组初始化

- 如果数组元素是基本类型int，其默认值为0；

```
int[] arr = new int[256];
```

```
arr[6] = 100;
```

- 数组元素是引用类型，默认值为null，使用之前必须先引用特定对象。

```
MyClass[] arr = new MyClass[10];
```

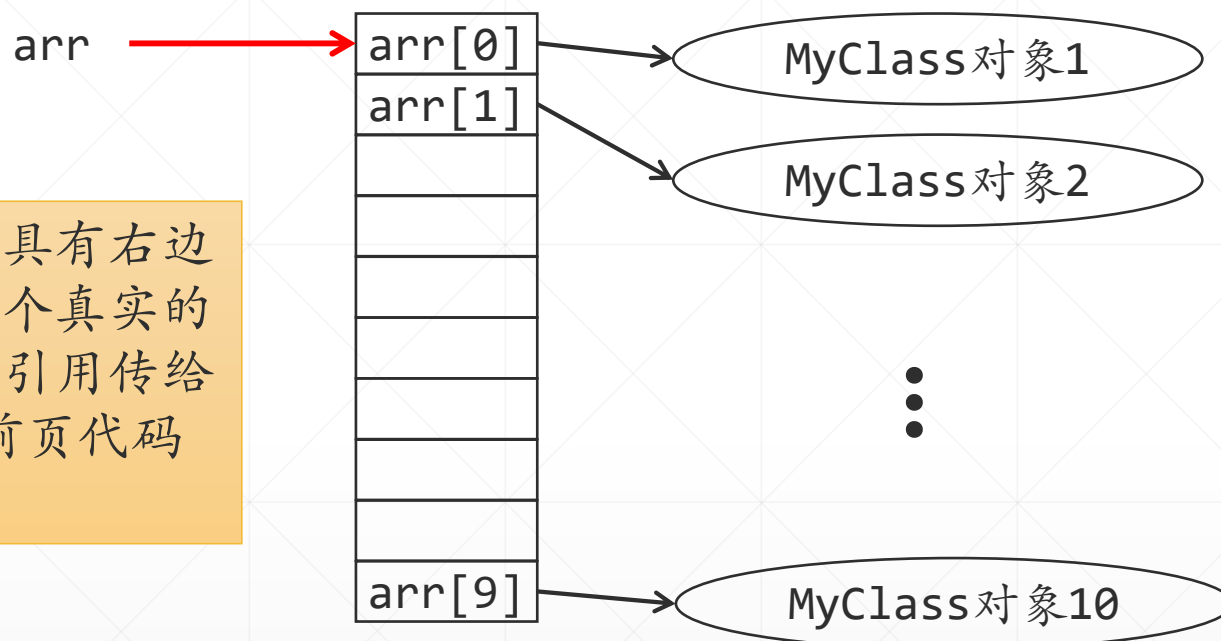
```
arr[7].value = 32;
```



这句在运行时将引发`java.lang.NullPointerException`，因为尝试访问一个还不存在的对象，这是一种常见的错误。

引用类型数组的内存布局

一个能正常访问的对象数组，应该具有右边的内存模型，每个数组元素引用一个真实的对象。如果没有显式地把某个对象引用传给数组元素，则其值为null，这正是前页代码会引发NullPointerException的原因。



知道了这个原因，那么，怎么改代码就很清楚了，留给大家作为练习。（用于练习的测试代码见[NullPointerExceptionExample.java](#)）

再次强调



- 对于数组元素是引用类型的数组，在访问数组中元素之前，需要创建相应的对象并让数组元素引用它！
- 另外，数组本身也是一个对象，数组变量引用数组对象
- 请牢记引用类型数组的内存布局模型，区分引用数组对象的数组变量，和引用外部对象的引用型数组元素，这两者的区别。

访问数组元素

数组元素可以通过下标直接定位，如果需要遍历，则通过循环语句实现，注意有两种循环遍历方式：

```
int[] arr = new int[10];  
//使用传统的for循环访问数组  
for (int i = 0; i < arr.length; i++)  
    arr[i] = i;  
  
//使用foreach循环访问数组  
for (var value : arr) {  
    System.out.print(value + ",");  
}
```

VisitArray.java

当用循环语句遍历访问数组元素时，注意

(1) 下标不能小于0

(2) 下标必须小于最大的数组元素数目
否则，对于非法的数组下标，Java会抛出异常
`ArrayIndexOutOfBoundsException`。
要处理这个异常，需要使用try...catch语句，
后面课程会详细介绍其语法

关于foreach访问数组

```
//使用foreach循环访问数组
for (var value : arr) {
    System.out.print(value + ",");
}
```

foreach循环是JDK 5.0引入的，主要用于顺序访问集合类型（数组也算集合类型）。

在循环变量中可以使用var定义循环变量（JDK10的特性），早期版本中循环变量必须显式指定为具体的数据类型。

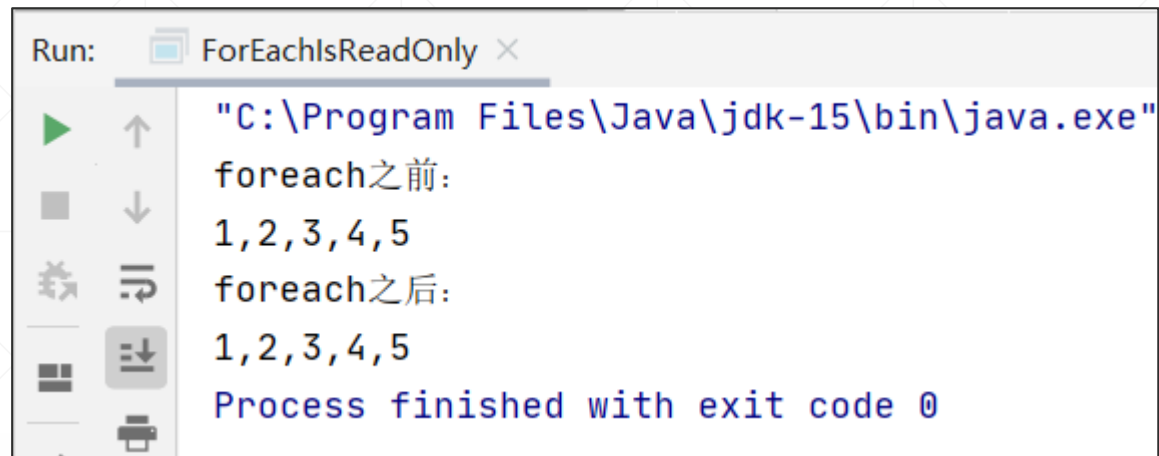
注意事项：

1. 循环变量必须在“（）”中定义
2. 集合变量可以是数组或任何一个实现了Iterable接口的集合类型。

什么叫接口？不要着急，后面课程介绍。

foreach循环的只读特性

```
public class ForEachIsReadOnly {  
    public static void main(String[] args) {  
        int[] arr={1,2,3,4,5};  
        System.out.println("foreach之前: ");  
        printArray(arr);  
        //在foreach循环中修改数值  
        for(int value : arr){  
            value *=2;  
        }  
        System.out.println("\nforeach之后: ");  
        printArray(arr);  
    }  
    //此方法将数组的内容转换为字符串，然后输出  
    static void printArray(int[] arr) {...}  
}
```



Run: ForEachIsReadOnly x

"C:\Program Files\Java\jdk-15\bin\java.exe"

foreach之前:
1,2,3,4,5
foreach之后:
1,2,3,4,5
Process finished with exit code 0

在foreach循环中不要修改被访问集合（包括数组）的内容，这个原则在开发中应该遵守。

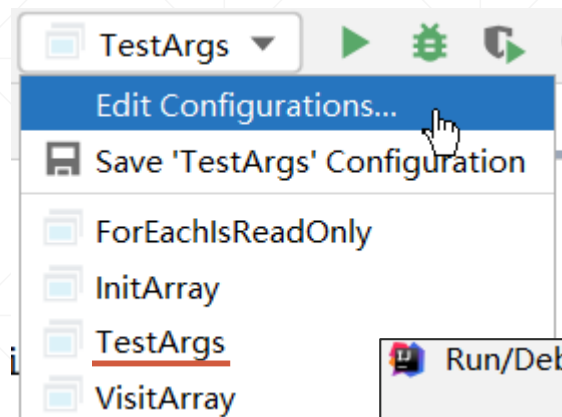
main方法的参数

main方法的参数，被当成是一个字符串数组，可以被循环遍历.....

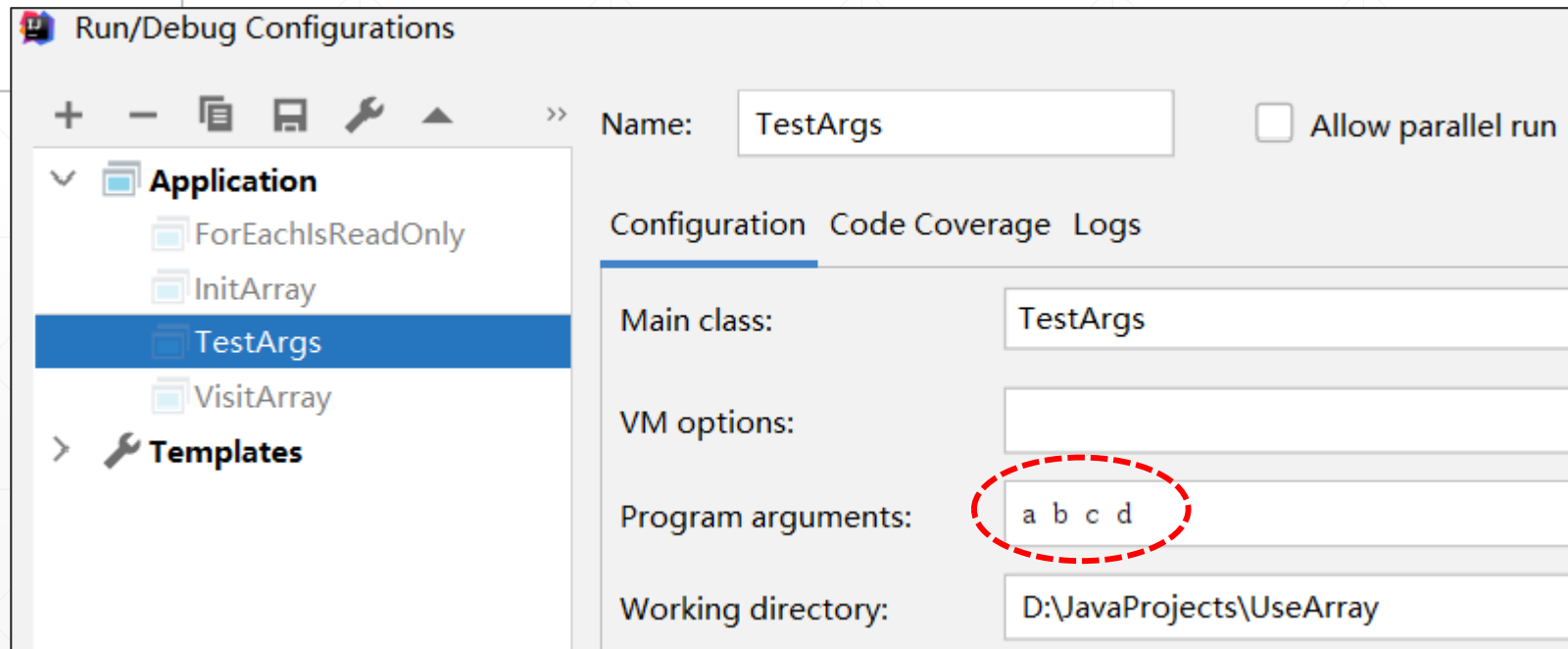
```
public static void main(String[] args) {  
    System.out.println(args.length);  
    for (String arg : args) {  
        System.out.println(arg);  
    }  
}
```

TestArgs.java

在IntelliJ中指定程序运行参数



先运行一下TestArgs程序，IntelliJ就会为它创建一个**运行/调试配置**环境，然后，你就可以修改这一配置，给它指定特定的参数值。



数组作为方法的参数

- 两种方法参数的传递方式
 - Pass-by-value（按值传送）：用于原始数据类型
 - Pass-by-reference（按引用传送）：对象都按引用传送
- 数组类型的方法参数按引用传递。

数据作为方法参数示例代码



阅读并运行示例PassArray.java，观察并分析程序输出的结果，小结，然后与下页幻灯片所讲的内容进行对照。

```
public static void modifyArray(int[] b) {  
    for (int j = 0; j < b.length; j++)  
        b[j] *= 2;  
}
```

```
int[] arr = new int[ 24 ];  
.....  
modifyArray( arr );
```

PassArray.java程序的输出



```
The values of the original array are:  
  1   2   3   4   5  
  
Effects of passing array element call-by-value:  
a[3] before modifyElement: 4  
a[3] after modifyElement: 4  
  Effects of passing entire array by reference  
  
The values of the modified array are:  
  2   4   6   8  10
```

按引用传递与按值传送方法参数的最大区别在于：

- 使用前者时，如果方法中有代码更改了数组元素的值，实际上是直接修改了原始的数组元素。
- 使用后者则没有这个问题，方法体中修改的仅是原始数组元素的一个副本。

数组作为实参，其实本质上也是“传值”

```
public static void main(String[] args) {  
    int[] oneArr = {1, 2, 3, 4};  
    System.out.println("Before:" + Arrays.toString(oneArr));  
    //Before:[1, 2, 3, 4]  
    processArr(oneArr);  
    //After:[2, 2, 3, 4]  
    System.out.println("After:" + Arrays.toString(oneArr));  
}
```

```
static void processArr(int[] arr) {  
    arr[0] *= 2;  
    int[] otherArr = {4, 5, 6};  
    //让arr引用另外一个数组  
    arr = otherArr;  
}
```

示例：ArrayAsArgument.java

从这个示例可以看出，以数组作为参数，其实是“数组变量”所保存内容的“值复制”，因此，在方法里面的arr，与外部传入的oneArr，就不是一回事，是独立的变量，所以，调用processArr方法之后，输出不会是“[4,5,6]”，而是[2,2,3,4]。

多维数组

大于一维的数组，称为“多维数组”，以下代码定义了一个二维数组，使用类似的方法，可以定义更高维的数组：

```
int a[][] = { { 1, 2 }, { 3, 4 } };
```

Java中的多维数组，可以是“不整齐”的！

```
int b[][];  
b = new int[ 2 ][ ];    // allocate rows  
b[ 0 ] = new int[ 5 ];  // allocate row 0  
b[ 1 ] = new int[ 3 ];  // allocate row 1
```

其实“没有”多维数组



多维数组本质上就是“一维”数组，只不过这个数组的元素又是一个“数组”罢了，由此可以不断地迭代，最终得到一个任意维数的数组。



`TestTwoDimension.java`示例展示了二维数组的用法，请自行阅读，此处不再赘述。

数组拷贝

JDK 中针对数组提供了一些现成的方法，比如使用 `arraycopy` 方法可以方便地复制一个数组：

```
int[] from = {0,1,2,3,4,5,6,7,8,9};  
int[] to = new int[10];  
System.arraycopy(from,0,to,0,10);  
for(int element:to)  
    System.out.println(element);
```

Arrays类中的静态方法

//定义一个a数组

```
int[] a = new int[]{3, 4, 5, 6};
```

//定义一个a2数组

```
int[] a2 = new int[]{3, 4, 5, 6};
```

TestArrays.java

JDK中有一个Arrays类，封装了一些与数组相关的静态方法，这些静态方法可以完成诸如查找、复制、比较等各种数组常用功能。

//a数组和a2数组的长度相等，每个元素依次相等，将输出true

```
System.out.println("a数组和a2数组是否相等: " + Arrays.equals(a, a2));
```

//通过复制a数组，生成一个新的b数组

```
int[] b = Arrays.copyOf(a, newLength: 6);
```

```
System.out.println("a数组和b数组是否相等: " + Arrays.equals(a, b));
```

//输出b数组的元素，将输出[3, 4, 5, 6, 0, 0]

```
System.out.println("b数组的元素为: " + Arrays.toString(b));
```

//将b数组的第3个元素（包括）到第5个元素（不包括）赋为1

```
Arrays.fill(b, fromIndex: 2, toIndex: 4, val: 1);
```

//输出b数组的元素，将输出[3, 4, 1, 1, 0, 0]

```
System.out.println("b数组的元素为: " + Arrays.toString(b));
```

//对b数组进行排序

```
Arrays.sort(b);
```

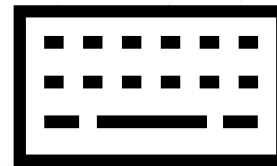
//输出b数组的元素，将输出[0, 0, 1, 1, 3, 4]

```
System.out.println("b数组的元素为: " + Arrays.toString(b));
```

课后作业



巩固练习



随机生成10个整数，用它们填充一个数组，首先显示数组内容，接着计算数组元素的“和”、“平均值”与“方差”，显示结果。

巩固



- 以下代码的输出结果是什么？为什么会有这个结果？

```
int[] a = {5, 7 , 20};  
int[] b = new int[4];  
System.out.println("b数组的长度为： " + b.length);  
b = a;  
System.out.println("b数组的长度为： " + b.length);
```

测试用代码：ArrayInRam.java

阅读程序能力训练



- 阅读程序WhatDoesThisDo.java, 解释程序所完成的功能
- 阅读程序WhatDoesThisDo2.java, 解释程序所完成的功能

动手动脑



《数据结构》是计算机专业的核心课程，这门课中介绍了许多排序方法，现在请使用Java编写一个示例，实现著名的“冒泡排序”，程序的输出如下所示：

```
Data items in original order
  2  6  4  8 10 12 89 68 45 37

Data items in ascending order
  2  4  6  8 10 12 37 45 68 89
```



请务必自己动手完成这个程序，然后再对比参考答案示例代码（见下页PPT），体会编程的一些基本技巧。

“冒泡排序”参考答案



- 参看BubbleSort.java
- 请仔细阅读示例代码，注意这个例子是如何在类中进行模块划分的（设计了哪几个方法，每个方法完成什么功能？），以后在自己的代码中有意识地应用这种代码组织方式。

数据的查找



《数据结构》还介绍了许多数据查找算法，最简单的数据查找方式是“**线性查找**”，即从头开始顺序查找，适合于非有序的数组，对于大型的数组，效率较低。



当数组中的元素已经有序时，“**二分查找**”效率更高。所谓二分查找，就是先拿要找的数与数组中间的那个数比，如果大于中间数，就到数组后半部分去找（否则，就到前半部分去找），与数组后半部分的“中间数”比，然后重复以上流程。

动手动脑



1

编写一个程序，随机生成一个数值（int/float/double）数组，排序，之后对它进行线性和二分查找。

2

使用搜索引擎，查找如何计算Java代码执行时间的方法，然后，对比测试一下“线性查找”和“二分查找”两种算法的执行时间，回答以下问题：

- (1) “二分查找”真的比“线性查找”快吗？
- (2) 如果真的快，快多少？
- (3) 不同算法的执行速度，与数据量有关系吗？
- (4) 还有没有更快的数据查找方法？翻翻《数据结构》教材，然后试着编写代码测试一下。

动手动脑



阅读QiPan.java示例程序

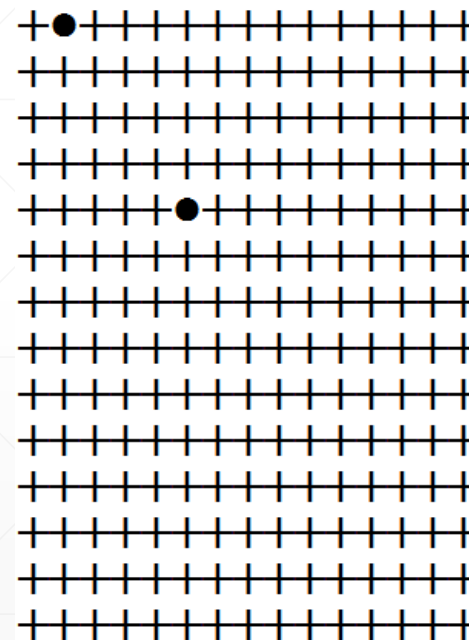
了解如何利用二维数组和循环语句绘制五子棋盘。

这个程序还是有很多简陋之处的，比如，要求用户严格按照预定格式输入坐标，不能区分黑棋和白棋等等。

你能想办法完善它吗？

请输入您下棋的座标，应以x,y的格式：

5, 6



动手动脑



1

请编写一个程序将一个整数转换为汉字读法字符串。比如“1123”转换为“一千一百二十三”。

2

更进一步，能否将数字表示的金额改为汉字表达？比如将“¥123.52”转换为“壹佰贰拾叁元伍角贰分”。

参考答案：Num2Rmb.java

迎接挑战!



前面几讲介绍过JDK所提供的BigInteger能完成大数计算，如果不用它，直接使用数组表达大数，你能实现相同的功能吗？

要求：

- (1) 用你的大数类实现加和减两个功能
- (2) 阅读BigInteger类源码，弄清楚它是使用什么算法实现加减乘除四种运算的？
- (3) 通过互联网查找大数运算的相关资料，给你的大数类添加乘、除、求阶乘等其它功能。

学以致用



编写一个简单的英文词频统计程序：

此程序读入一篇英文文章，自动抽取出其中的所有词，统计每个词的频数，然后按照从高到低顺序显示出前10个单词。



可以利用StringTokenizer类，如果你觉得这个类不能满足你的需求，那你能不能自己写一个？



延伸挑战：
能否尝试一下中英文混杂的文章的词频统计？